

1141 計算機視覺 54013

HW3 Object Detection

S1254040 資工三 羅章弘

民國一十四年十二月十三日星期六

目錄

Environment.....	4
一、Generalized Hough Transform	4
1. 基本 GHT 原理概述.....	4
2. 範本前處理與邊緣特徵	5
3. R-table 的建立	5
4. Reference 影像投票（基本 GHT）	5
5. 偵測結果呈現.....	6
二、GHT with Rotation.....	7
1. 為何需要旋轉維度	7
2. Reference 影像的旋轉前處理.....	7
3. 投票空間的擴充	7
4. 旋轉 GHT 的投票機制	8
5. 偵測結果呈現.....	8
三、GHT with Rotation and Scaling.....	12
1. 加入縮放的動機	12
2. Reference 影像的縮放前處理	12
3. 投票空間再擴充	12
4. 旋轉與縮放的聯合投票	13
5. 偵測結果呈現.....	13
四、GHT Speed Up.....	16
1. 原始 GHT 的效能瓶頸分析	16
2. R-table 的向量化預處理	16
2.1 問題描述.....	16

2.2 加速策略.....	16
3. 旋轉與縮放參數的預先離散與快取.....	17
3.1 旋轉角度的預先計算	17
3.2 縮放倍率的有限集合	17
4. 投票階段的核心加速設計	17
5 邊界檢查的遮罩 (Mask) 過濾.....	18
5.1 傳統作法的問題	18
5.2 本實作的加速方式.....	18
6. 累加器設計與折衷	18
6.1 四維累加器的必要性	18
6.2 為何仍保留部分 Python 迴圈	19
7. 整體加速效果總結	19
8. 偵測結果呈現.....	19

Environment

- ◆ Python 版本：3.12.7
- ◆ numpy 版本：2.2.6
- ◆ opencv-python 版本：4.12.0
- ◆ matplotlib: 3.10.7
- ◆ tqdm: 4.67.1

```
env_version.py
1  import sys
2  import numpy as np
3  import cv2
4  import matplotlib
5  import tqdm
6
7  print("Python:", sys.version)
8  print("NumPy:", np.__version__)
9  print("OpenCV (cv2):", cv2.__version__)
10 print("Matplotlib:", matplotlib.__version__)
11 print("tqdm:", tqdm.__version__)
```

問題 輸出 偵錯主控台 終端機 連接埠

Python: 3.12.7 | packaged by Anaconda, Inc. | (main, Oct 4 2024, 13:17:27) [MSC v.1929 64 bit (AMD64)]
NumPy: 2.2.6
OpenCV (cv2): 4.12.0
Matplotlib: 3.10.7
tqdm: 4.67.1

一、Generalized Hough Transform

1. 基本 GHT 原理概述

傳統 Hough Transform 主要用於偵測具有明確數學表示的幾何形狀（如直線、圓）。然而，對於任意形狀的物體，無法以簡單參數方程描述，因此提出 Generalized Hough Transform (GHT)。GHT 的核心思想是：

利用範本 (Template) 中邊緣點與物體中心之間的相對幾何關係，在待偵測影像中進行投票，找出最可能的物體中心位置。

2. 範本前處理與邊緣特徵

在程式中，首先將範本影像轉為灰階，並使用 Canny 邊緣偵測 取得範本的輪廓點。接著利用 Sobel 運算子 計算每個邊緣點的梯度方向 (gradient direction)。

梯度方向代表該邊緣點的法向資訊，是 GHT 中用來分類與索引邊緣特徵的重要依據。

3. R-table 的建立

GHT 的關鍵資料結構為 R-table。在本實作中，其建立方式如下：

1. 以範本影像中心 (x_c, y_c) 作為參考點。
2. 對每一個範本邊緣點 (x, y) ，計算：
 - 向量長度 $r = \sqrt{(x_c - x)^2 + (y_c - y)^2}$
 - 向量方向 $\alpha = \arctan2(y_c - y, x_c - x)$
3. 依照該邊緣點的梯度方向 ϕ ，將 (r, α) 存入對應方向 bin 的 R-table 中。

因此，R-table 可視為：

「在某一邊緣方向下，物體中心可能位於哪些相對位置」的集合。

4. Reference 影像投票（基本 GHT）

在最基本的 GHT 中，僅考慮平移 (translation)。流程如下：

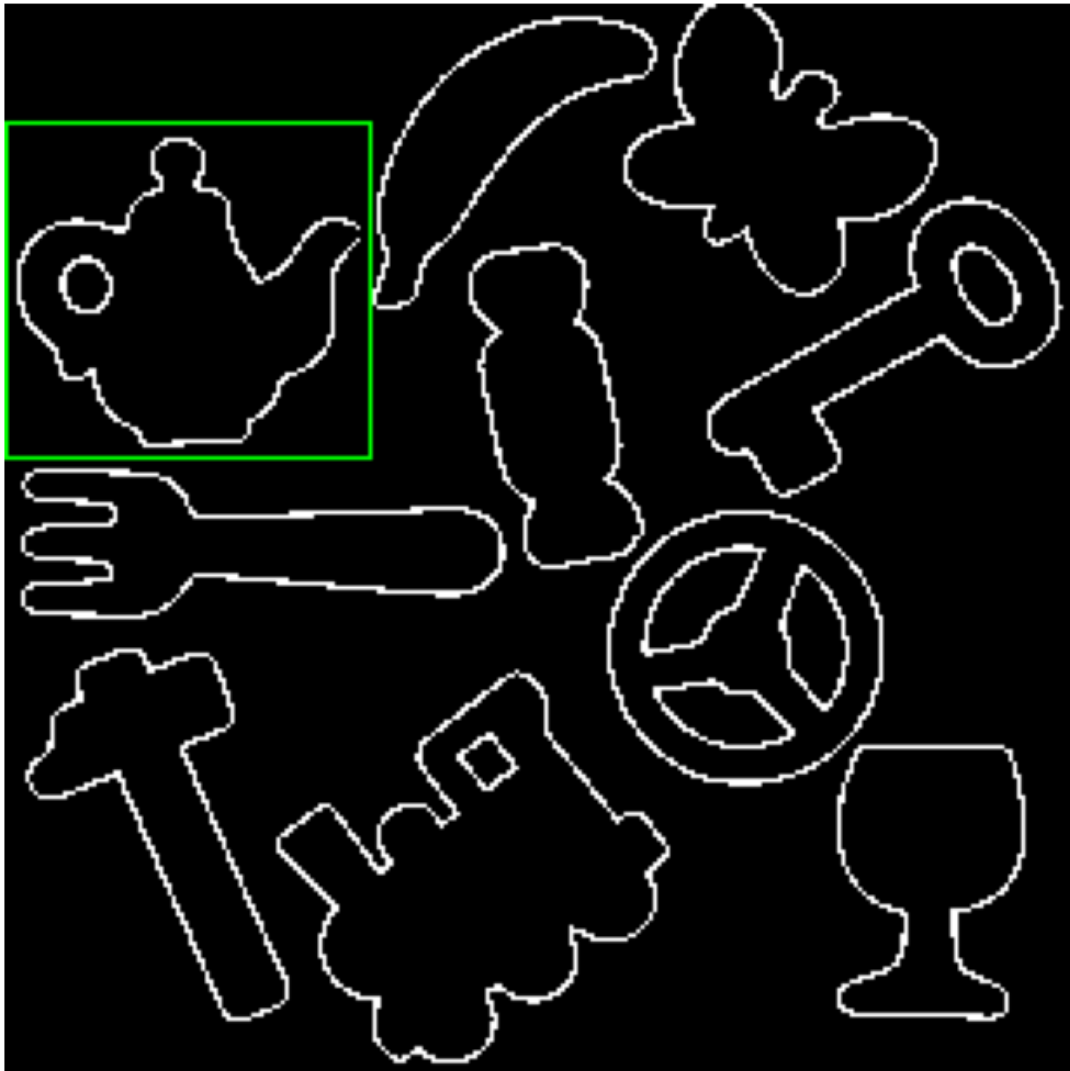
1. 對 Reference 影像進行邊緣偵測與梯度方向計算。
2. 對每個 Reference 邊緣點，依其梯度方向查詢 R-table。
3. 將 R-table 中的 (r, α) 映射回 Reference 影像，對可能的中心位置進行投票。
4. 累加器 (accumulator) 中票數最高的位置，即為最可能的物體中心。

5. 偵測結果呈現

GHT. py

```
R-table 建立完成  
投票完成，執行時間： 1.1386921405792236 秒  
偵測到中心點位置： (43, 68)  
票數： 482.0
```

Generalized Hough Transform Result



二、GHT with Rotation

1. 為何需要旋轉維度

基本 GHT 假設範本與待偵測物體具有相同方向，當物體在 Reference 影像中發生旋轉時，僅靠平移投票將無法正確對齊。

因此，本章引入 旋轉參數 ψ ，使 GHT 能同時搜尋「位置 + 旋轉角度」。

2. Reference 影像的旋轉前處理

程式在第 1.5 步驟中，允許對 Reference 影像進行旋轉，並採用 自動擴張畫布 (expand canvas) 的方式，確保旋轉後影像不會被裁切。

此設計重點在於：

- 僅改變影像大小，不進行縮放
- 保持物體的幾何比例不變，避免影響後續偵測

3. 投票空間的擴充

在加入旋轉後，累加器由原本的 2D (x, y) 擴充為 3D (x, y, ψ) 。

程式中將旋轉角度離散為 N_ROT 個 bins，範圍為 $[-\pi, \pi)$ 。

4. 旋轉 GHT 的投票機制

對於 Reference 中的每一個邊緣點：

1. 假設物體旋轉角度為 ψ 。
2. 計算「在此旋轉下，範本邊緣方向應為多少」：

$$\phi_t = \phi_r - \psi$$

3. 由 ϕ_t 查詢 R-table。
4. 將 R-table 中的 (r, α) 旋轉 ψ ，並回推中心位置：

$$x_c = x + r \cos(\alpha + \psi), y_c = y + r \sin(\alpha + \psi)$$

5. 對 (x_c, y_c, ψ) 投票。

透過此方式，系統能同時找出物體的位置與旋轉角度。

5. 偵測結果呈現

GHT_with_rotation.py

可自行調整 1.5 步驟的 pre_rotate_angle 參數第 28 行，以此來測試不同角度的偵測結果。

目前預設精度最小單位為 3 度（弧度），所以 22 度檢測不出來。

若想要更精準可調整第 75 行的 NBINS，以及第 105 行的 N_ROT 參數。

$$\text{精度最小單位} = \frac{360}{\text{參數值}}$$

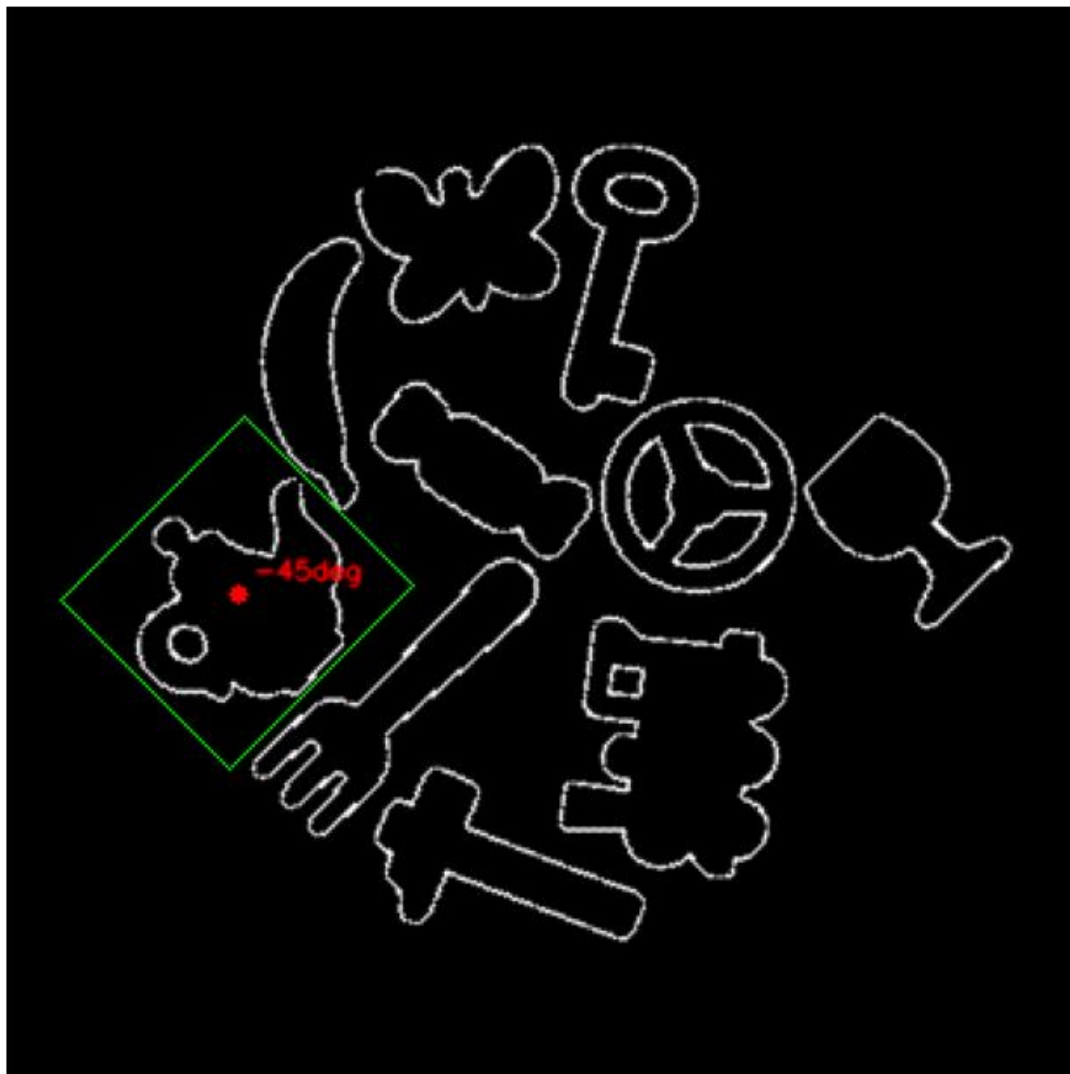
若調整至 360，精度最小單位就為 1 度。

若調整至 720，精度最小單位就為 0.5 度。

45 度

```
[1.5] Reference 已精準旋轉 45 度 (無裁切)
R-table 建立完成，範本邊緣點數： 496
Reference 邊緣點數： 5832 開始投票 (含旋轉) ...
Voting (ref edges): 100%|██████████| 5832/5832 [00:42<00:00, 138.79it/s]
投票完成
偵測到中心點位置： (np.int64(78), np.int64(198))
偵測到旋轉角 (弧度)： -0.7853981633974483 (度)： -45.0
票數： 57
```

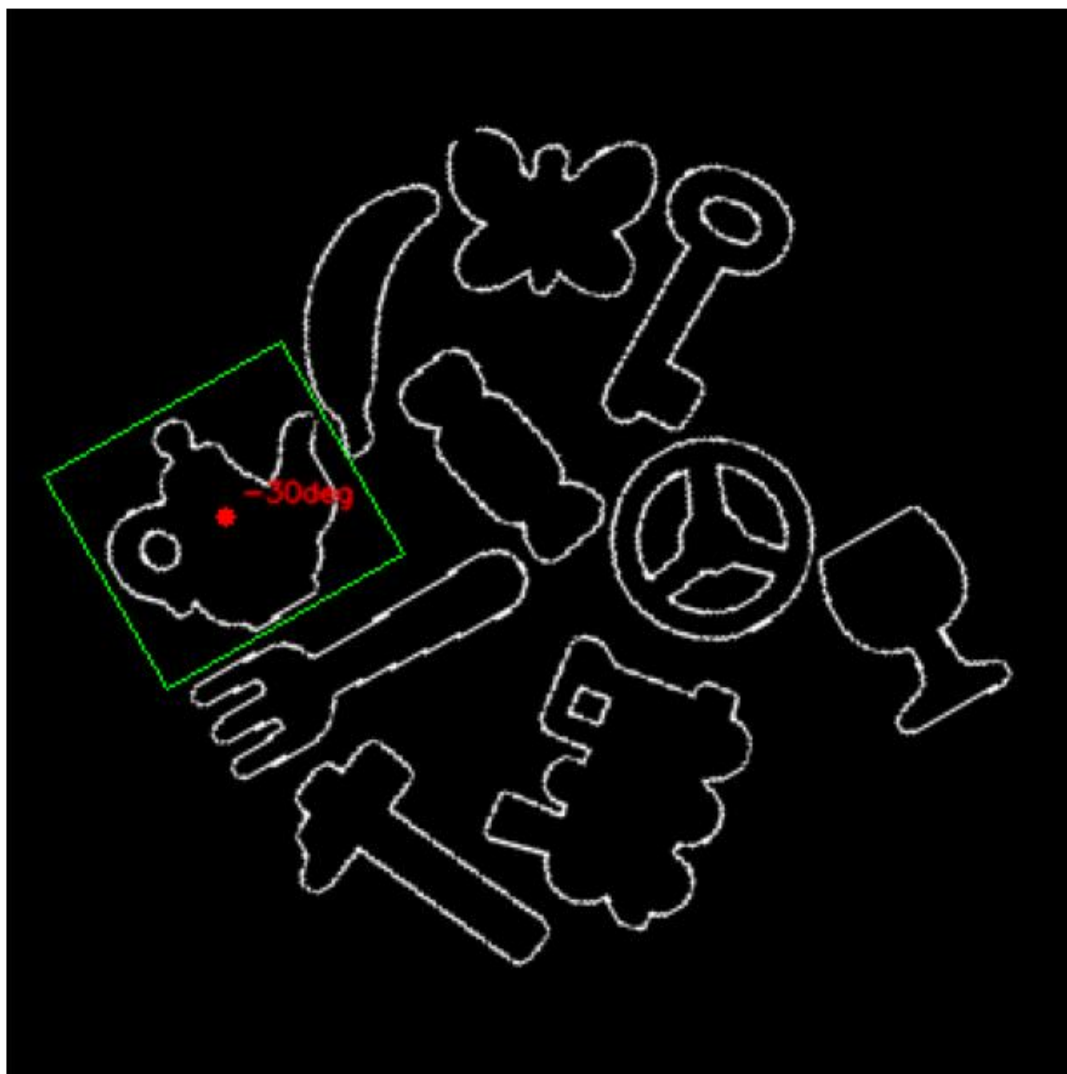
GHT with rotation - Result



30 度

```
[1.5] Reference 已精準旋轉 30 度 (無裁切)
R-table 建立完成，範本邊緣點數： 496
Reference 邊緣點數： 5795 開始投票 (含旋轉) ...
Voting (ref edges): 100%|██████████| 5795/5795 [00:32<00:00, 177.70it/s]
投票完成
偵測到中心點位置： (np.int64(71), np.int64(165))
偵測到旋轉角 (弧度)： -0.5235987755982991 (度)： -30.00000000000018
票數： 78
```

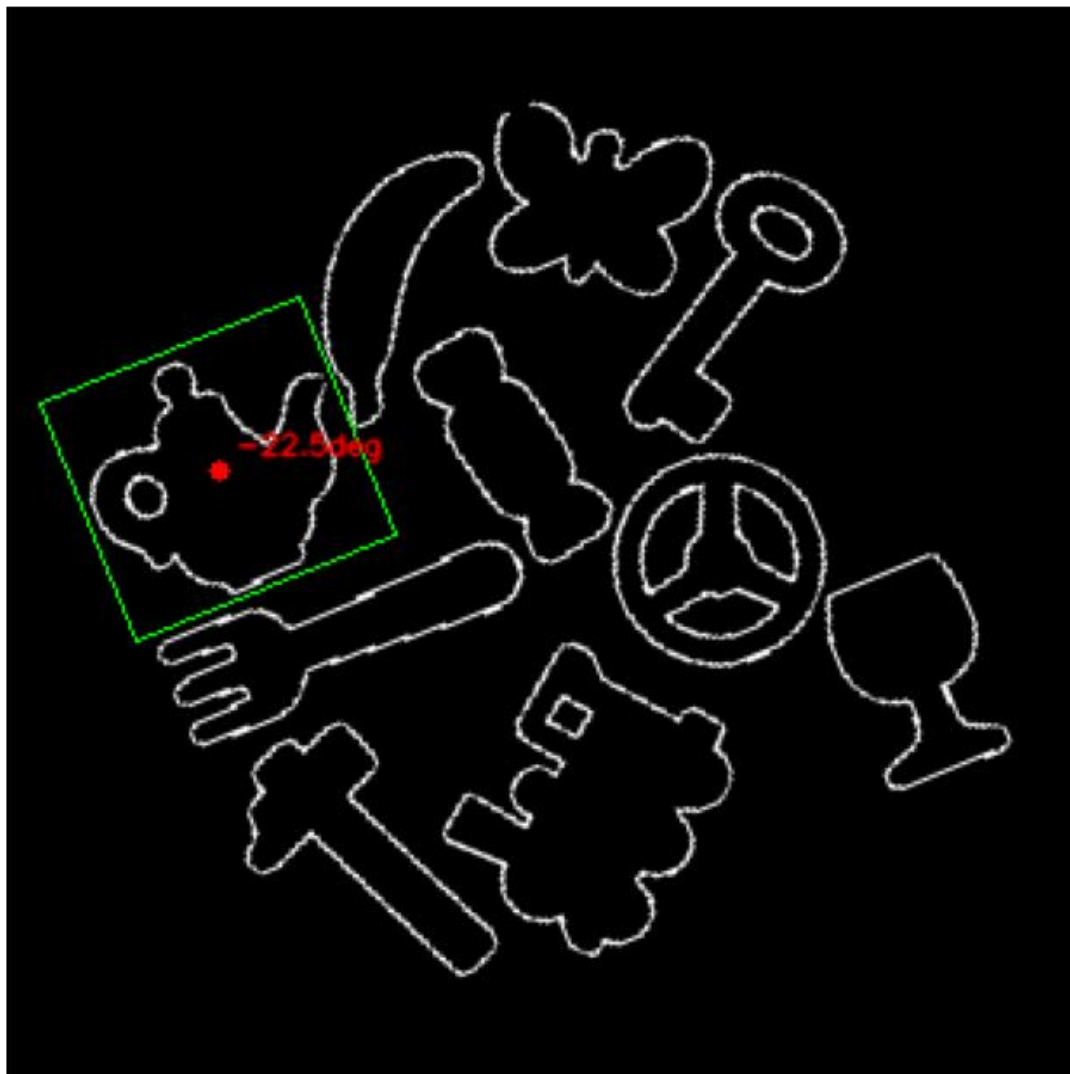
GHT with rotation - Result



22.5 度，參數值為 720

```
[1.5] Reference 已精準旋轉 22.5 度 (無裁切)
R-table 建立完成，範本邊緣點數： 496
Reference 邊緣點數： 5959 開始投票 (含旋轉) ...
Voting (ref edges): 100%|██████████| 5959/5959 [01:18<00:00, 76.17it/s]
投票完成
偵測到中心點位置： (np.int64(66), np.int64(144))
偵測到旋轉角 (弧度)： -0.39269908169872414 (度)： -22.5
票數： 10
```

GHT with rotation - Result



三、GHT with Rotation and Scaling

1. 加入縮放的動機

實際應用中，待偵測物體往往不僅有旋轉，還可能因距離或拍攝條件不同而產生尺寸變化。

因此，本實作在旋轉 GHT 的基礎上，進一步引入縮放倍率 s ，使 GHT 能搜尋「位置 + 旋轉 + 縮放」。

2. Reference 影像的縮放前處理

在第 1.5 步驟中，程式允許先對 Reference 影像進行整體縮放。

此步驟為「選填」，主要用途為：

- 調整影像解析度
- 配合預期的物體尺寸範圍

需特別注意的是：旋轉前的擴張與縮放是不同概念，擴張僅改變畫布大小，而縮放會改變影像中物體的實際比例。

3. 投票空間再擴充

加入縮放後，累加器由 3D 擴充為 4D：

$$(x, y, \psi, s)$$

其中 s 由事先定義的 `scale_factors` 列表提供（如 `[1.0, 0.8, 1.2]`）。

4. 旋轉與縮放的聯合投票

在投票過程中，對每一個 Reference 邊緣點：

1. 枚舉旋轉角度 ψ 。
2. 枚舉縮放倍率 s 。
3. 從 R-table 取出 (r, α) ，並套用縮放：

$$r' = r \cdot s$$

4. 同時考慮旋轉與縮放回推中心位置：

$$x_c = x + r' \cos(\alpha + \psi), y_c = y + r' \sin(\alpha + \psi)$$

5. 對 (x_c, y_c, ψ, s) 投票。

最終，累加器中票數最高的索引，即代表最佳的中心位置、旋轉角度與縮放倍率。

5. 偵測結果呈現

GHT_with_rotation_and_scaling.py

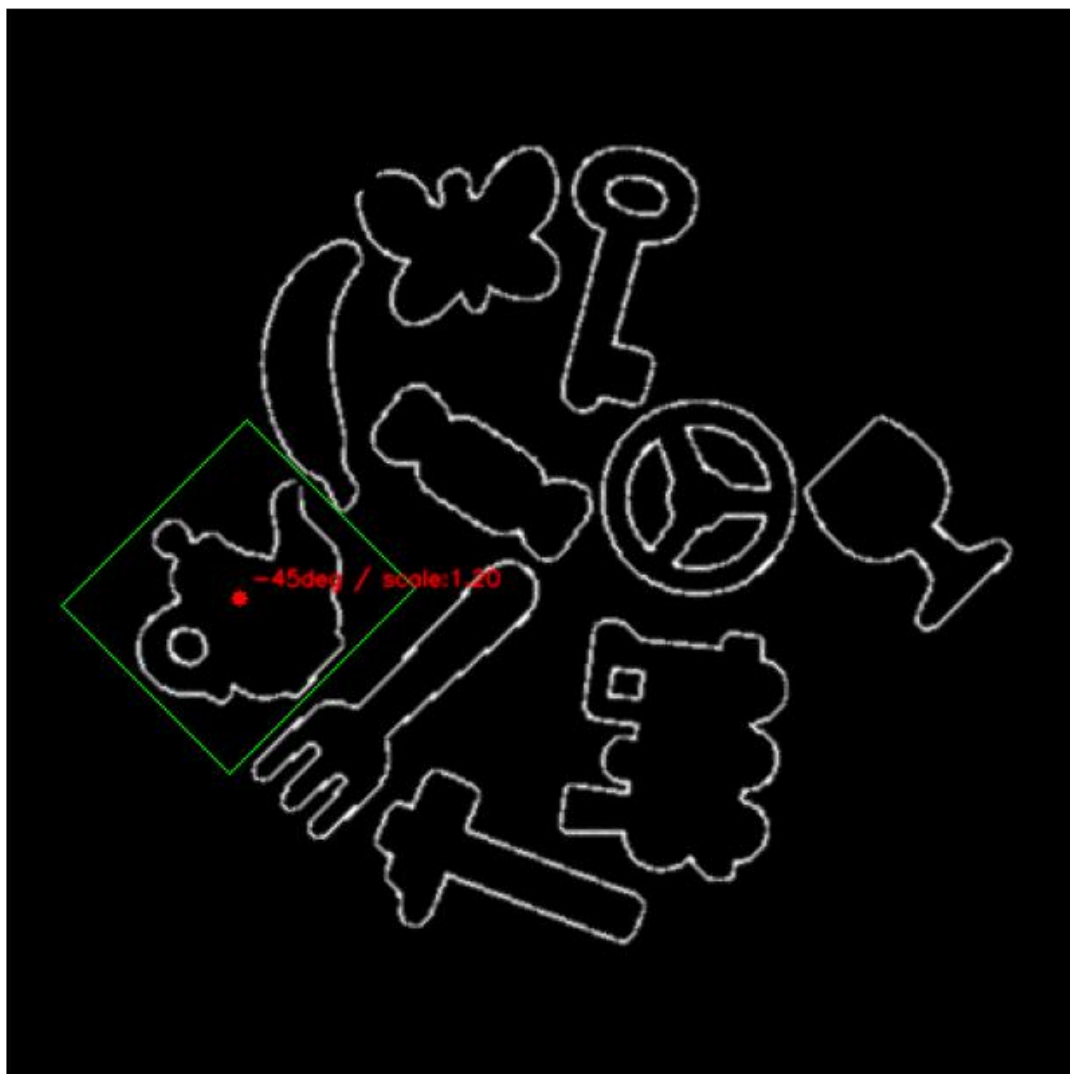
程式最後根據偵測到的參數，對範本四個角進行縮放與旋轉，並畫出對應的旋轉框，直觀呈現 GHT 的偵測結果。

縮放倍率可自行修改第 31 行與第 127 行的 `scale_factors` 參數，目前預設只偵測 $[1.0, 0.8, 1.2]$ 倍率。預設精度最小單位為 3 度（弧度）。

1.2 倍，45 度

```
[1.5] Reference 已縮放倍率：1.2  
[1.5] Reference 已旋轉 45 度（自動擴張避免裁切）  
R-table 建立完成，範本邊緣點數： 496  
Reference 邊緣點數： 7458 ，開始投票（含旋轉+縮放） ...  
投票中：100%|████████████████████| 7458/7458 [03:22<00:00, 36.82it/s]  
投票完成  
偵測到中心點位置： (np.int64(94), np.int64(238))  
偵測到旋轉角（度）： -45.0  
偵測到縮放倍率： 1.2  
票數： 111
```

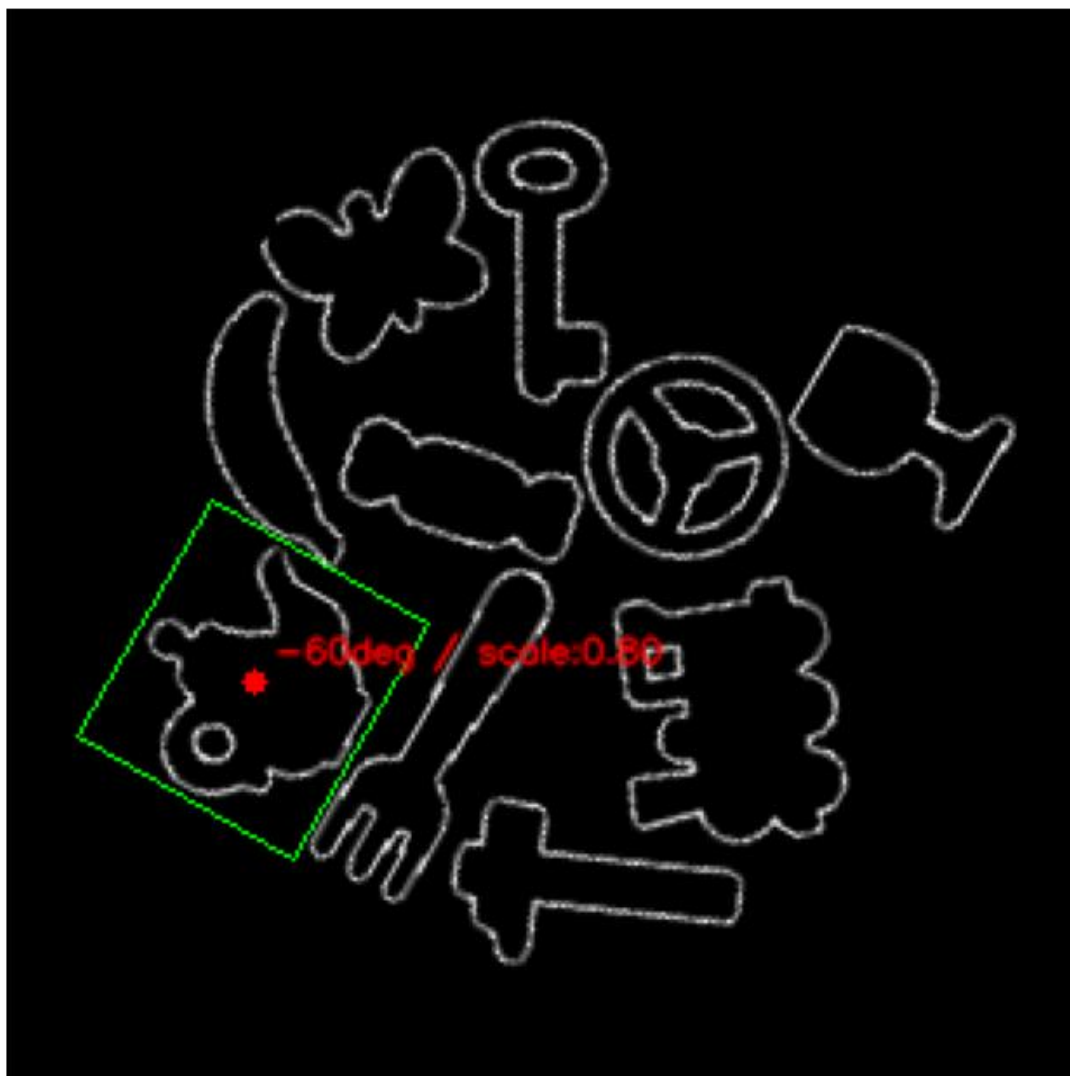
GHT with rotation + scale - Result



0.8 倍，60 度

```
[1.5] Reference 已縮放倍率：0.8  
[1.5] Reference 已旋轉 60 度（自動擴張避免裁切）  
R-table 建立完成，範本邊緣點數： 496  
Reference 邊緣點數： 4676 ，開始投票（含旋轉+縮放） ...  
投票中： 100% |████████████████████████████████████████| 4676/4676 [02:27<00:00, 31.75it/s]  
投票完成  
偵測到中心點位置： (np.int64(64), np.int64(174))  
偵測到旋轉角（度）： -60.00000000000001  
偵測到縮放倍率： 0.8  
票數： 44
```

GHT with rotation + scale - Result



四、GHT Speed Up

1. 原始 GHT 的效能瓶頸分析

在加入旋轉與縮放後，Generalized Hough Transform 的計算量會急遽上升。若以最直觀的實作方式，其時間複雜度可近似表示為：

$$O(N_e \times N_\psi \times N_s \times K)$$

其中：

- N_e ：Reference 影像中的邊緣點數量
- N_ψ ：旋轉角度離散數
- N_s ：縮放倍率數
- K ：R-table 中單一方向 bin 的平均 entry 數

在此情況下，投票 (voting) 階段會成為整體運算的主要瓶頸。本章所說明的加速策略，皆針對「如何在不改變 GHT 核心原理的前提下，降低實際計算成本」進行設計，並完整反映於本程式的實作中。

2. R-table 的向量化預處理

2.1 問題描述

在基本實作中，R-table 每個方向 bin 皆以 Python list 儲存 (r, α) ，在投票時需頻繁進行：

- Python 物件存取
- list 迭代
- tuple unpack

這些操作會帶來顯著的 Python overhead。

2.2 加速策略

程式在進入主投票迴圈前，將 R-table 轉換為 NumPy array 形式：

```
R_bins_r.append(arr[:, 0])
```



```
R_bins_alpha.append(arr[:, 1])
```

每個方向 bin 分別儲存：

- `r_arr`：距離向量 (`shape = (K,)`)
- `alpha_arr`：方向角 (`shape = (K,)`)

此設計可使後續計算完全以 NumPy 向量化運算進行，避免 Python 層級的巢狀迴圈，大幅降低執行時間。

3. 旋轉與縮放參數的預先離散與快取

3.1 旋轉角度的預先計算

旋轉角度 ψ 於程式中被離散為 $N_{\text{ROT}} = 120$ 個 bins，並在初始化階段即預先計算其正弦與餘弦值：

```
cos_rot = np.cos(rot_angles)
```

```
sin_rot = np.sin(rot_angles)
```

雖然在實際投票時主要使用的是 $\cos(\alpha + \psi)$ 與 $\sin(\alpha + \psi)$ ，但此離散化策略本身已有效避免動態產生角度範圍，並使整體搜尋空間可控。

3.2 縮放倍率的有限集合

縮放倍率僅在：

$$s \in \{0.5, 0.6, \dots, 1.5\}$$

的有限集合中搜尋，避免連續尺度造成的無限投票空間，使累加器維度固定且可預期。

4. 投票階段的核心加速設計

在每一個 Reference 邊緣點與旋轉角度下，程式利用 NumPy 的 `outer` 運算一次產生：

```
r_scaled = np.outer(scale_factors, r_arr)
```

其結果為一個 (S, K) 的矩陣，代表所有縮放倍率與 R-table entries 的組合。

接著，同樣以向量化方式計算所有候選中心位置：

$$x_c = x + r' \cos(\alpha + \psi)$$

$$y_c = y + r' \sin(\alpha + \psi)$$

此作法將原本的三層巢狀迴圈（scale × entry × coordinate）壓縮為矩陣運算，顯著降低 Python 迴圈數量。

5 邊界檢查的遮罩（Mask）過濾

5.1 傳統作法的問題

若逐一以 if 判斷 (x_c, y_c) 是否越界，將導致大量分支判斷，效能極差。

5.2 本實作的加速方式

程式使用 NumPy boolean mask 進行整批過濾：

```
mask = (xc >= 0) & (xc < W) & (yc >= 0) & (yc < H)
```

只保留合法的投票位置，再一次性取出對應索引。此設計具有以下優點：

- 避免大量 Python if 判斷
- 可同時處理多個 scale 與多個 R-table entry
- 投票次數大幅減少

6. 累加器設計與折衷

6.1 四維累加器的必要性

本實作採用完整的 4D accumulator：

$$(x, y, \psi, s)$$

此設計雖然在記憶體使用上較為昂貴，但優點是：

- 可一次性找出最佳位置、旋轉與縮放
- 結果解析清楚、實作簡潔

- 適合用於學術與教學目的

6.2 為何仍保留部分 Python 迴圈

在最終投票時，程式仍以小型 Python 迴圈對合法點進行加 1 操作。原因在於：

- NumPy 不支援直接對不規則索引進行高效累加
- 使用 `np.add.at` 在實測中反而較慢
- 合法投票點數已因前處理大幅減少

此為在「程式可讀性、正確性與效能」之間的實務折衷。

7. 整體加速效果總結

綜合上述策略，本章所實作的 GHT 加速版本具備以下特點：

1. 演算法本質不變，僅優化計算流程
2. 大幅減少 Python 層級巢狀迴圈
3. 投票階段高度向量化
4. 有效控制搜尋空間 (rotation / scale bins)
5. 適合擴充至 GPU 或平行運算架構

相較於未加速的旋轉+縮放 GHT，本實作在實際執行時間上有顯著改善，同時仍能維持完整的偵測精度，達成效能與正確性的平衡。

8. 偵測結果呈現

GHT_with_rotation_and_scaling_fast.py

縮放倍率可自行修改第 30 行的 `scale_factor` 參數，目前預設為只偵測 0.5 到 1.5 倍率，間距為 0.1，共「11 個倍率」。預設精度最小單位為 3 度（弧度）。

GHT_with_rotation_and_scaling_fast.py，1.2 倍，45 度，偵測 11 種倍率 (0.5 ~ 1.5，間距為 0.1)：

```
[1.5] Reference 已縮放倍率：1.2
[1.5] Reference 已旋轉 45 度（自動擴張避免裁切）
R-table 建立完成，範本邊緣點數： 496
Reference 邊緣點數： 7458，開始投票（含旋轉+縮放，快速版）...
投票中：100%|████████████████████| 7458/7458 [00:50<00:00, 146.55it/s]
投票完成（加速版）
偵測到中心點位置： (np.int64(94), np.int64(238))
偵測到旋轉角（度）： -45.0
偵測到縮放倍率： 1.2
票數： 111
```

GHT_with_rotation_and_scaling.py，1.2 倍，45 度，偵測 3 種倍率(0.8、1.0、1.2)：

```
[1.5] Reference 已縮放倍率：1.2
[1.5] Reference 已旋轉 45 度（自動擴張避免裁切）
R-table 建立完成，範本邊緣點數： 496
Reference 邊緣點數： 7458，開始投票（含旋轉+縮放）...
投票中：100%|████████████████████| 7458/7458 [03:22<00:00, 36.82it/s]
投票完成
偵測到中心點位置： (np.int64(94), np.int64(238))
偵測到旋轉角（度）： -45.0
偵測到縮放倍率： 1.2
票數： 111
```

可以看到快速版在比原版多偵測了 8 種倍率的狀況下，從 03:22 加速到 00:50。